Algebra and Normalisation

Ohad Kammar

slides



course homepage



Zulip channel #Algebra and normalisation



Scottish Programming Languages and Verification Summer School (SPLV)
3–7 August 2026
University of Glasgow

THE UNIVERSITY of EDINBURGH informatics | lfcs Laboratory for Foundations of Computer Science | BayesCentre | Funded by: THE ROYAL SOCIETY

Normalisation in PL and Logic: what?

Goal

Identify equivalent syntactic representations

Example: equivalent programs

$x := 1;$
$x := 2$

vs

$x := 2$

vs

if $x > 0$
then $x := 2$
else $x := 2$

Example: algebraic expressions

$5 + x + (0 + (-6))$ vs $-1 + x$

Example: λ-terms (not today)

$(\lambda x : \mathbb{Z}.x + 1)(y - 3)$ vs $-2 + y$

Normalisation: why?

Proofs

Deal with fewer cases

Robust systems: type-checkers, compilers, and partial evaluators

Behave in the same way and compositionally for equivalent programs

Search and synthesis

Enumerate candidates over smaller search-space

Normalisation and rewriting

Blind Rewriting (BRew)

Temptation:

- Represent abstract syntax tree (AST)
- Apply all transformations
- Everywhere
- Blindly

BRew Caveats

- **Wasteful** in time & space (NB: **E-graphs**)
- **Chaotic** and **non-robust**: type errors after tiny tweaks to source
- **Syntax-sensitive** or **limited** (e.g.: $x + y$ vs $y + x$)

Normalisation and rewriting

Disclaimer!

Don't confuse **BRew** with **strategic** & **term rewriting**:

- Deep maths & time-tested properties:
  confluence, **strong normalisation**, **Knuth-Bendix completion**, ...
- Deep connections to foundations of **logic**, **mathematics**, & **computation**
  (e.g.: λ-calculus)

This course

Goal

relate **normalisation** and **modern algebra**

This course

Goal

relate **normalisation** and **modern algebra**

By the end of the course, you will be able to:

- Identify **free algebras** and **free extensions** with **normalisation** tasks.
- Phrase and prove **representation theorems**.
- Design **data-structures** for normalisation based on representation theorems.

This course comprises:

- 3 lecture hours.
- 3–6 independent self-study hours.
- 40 independent exercises hours.

So the majority of the learning will happen **after** SPLV.



course URL



Zulip URL

Course structure

Goal

relate **normalisation** and **modern algebra**

course URL

Zulip URL

- Part 1: universal algebra
  - Motivation
  - Universal algebra basics: signatures & algebras
  - Modern algebra: homomorphisms & representation theorems
  - Expression normalisation: free models & equational logic
  - Modern partial evaluation: free extensions
  - Categorical algebra basics
- Part 2: second-order multi-sorted algebra
- ~~Part 3: second-order normalisation~~

Universal algebra basics

template:	$(X_s)_{s \in \mathrm{Sort}}$ family of **carrier** sets	&	$\left(f_i : X_{s_1} \times \cdots \times X_{s_n} \to X_r\right)_{i \in I}$ family of **operations**	
instances: ($\mathrm{Sort} = \{\star\}$)	$\mathbb{Z} := \{\ldots, -2, -1, 0, 1, 2, \ldots\}$ integers	&	$(+) : \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}$	addition
			$0 : \mathbb{1} := \{\star\} \to \mathbb{Z}$	zero
			$(\cdot) : \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}$	multiplication
			$1 : \mathbb{1} \to \mathbb{Z}$	one

Universal algebra basics

template:	$(X_s)_{s \in \mathrm{Sort}}$ family of **carrier** sets	&	$\left(f_i : X_{s_1} \times \cdots \times X_{s_n} \to X_r\right)_{i \in I}$ family of **operations**	
instances:	$\mathbb{Z}$	&	$(+) : \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}$	addition
$(\mathrm{Sort} = \{\star\})$	integers		$0 : \mathbb{1} := \{\star\} \to \mathbb{Z}$	zero
$(\mathrm{Sort} = \mathbb{N} \times \mathbb{N})$	$(\mathbb{M}_{m \times n})_{m,n \in \mathbb{N}}$	&	$(\cdot) : \mathbb{M}_{m \times k} \times \mathbb{M}_{k \times n} \to \mathbb{M}_{m \times n}$	multiplication
	real matrices		$\mathrm{I}_n : \mathbb{1} \to \mathbb{M}_{n \times n}$	unit matrix

Universal algebra basics

template:	$(X_s)_{s \in \mathsf{Sort}}$ family of **carrier** sets	&	$\left(f_i : X_{s_1} \times \cdots \times X_{s_n} \to X_r\right)_{i \in I}$ family of **operations**	
instances: $(\mathsf{Sort} = \{\star\})$	$\mathbb{Z}$ integers	&	$(+) : \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}$ $0 : \mathbb{1} := \{\star\} \to \mathbb{Z}$	addition zero
$(\mathsf{Sort} = \mathbb{N} \times \mathbb{N})$	$\left(\mathbb{M}_{m \times n}\right)_{m,n \in \mathbb{N}}$ real matrices	&	$(\cdot) : \mathbb{M}_{m \times k} \times \mathbb{M}_{k \times n} \to \mathbb{M}_{m \times n}$ $I_n : \mathbb{1} \to \mathbb{M}_{n \times n}$	multiplication unit matrix
$\left(\mathbb{\Lambda}_A := \begin{pmatrix}\text{closed}\\ \lambda\text{-terms}\\ \text{of type } A\end{pmatrix} / \equiv_{\beta\eta}\right)_{A \in \mathsf{SimpleType}}$ combinatory logic		&	$(@) : \mathbb{\Lambda}_{A \to B} \times \mathbb{\Lambda}_A \to \mathbb{\Lambda}_B$ $I := (\lambda x : A.x) : \mathbb{1} \to \mathbb{\Lambda}_{A \to A}$ $K := (\lambda x : A, y : B.x) : \mathbb{1} \to \mathbb{\Lambda}_{A \to B \to A}$ $S := \left(\lambda x : A \to B \to C, y : A \to B, z : A.\right.$ $\left.x\ z (y\ z)\right) : \mathbb{1} \to \mathbb{\Lambda}_{(\ldots)}$	application combinators

$\vdots$

Universal algebra basics

Universal algebra

Study **common properties** of all such structures

(i.e.: universally applicable)

We'll do it in two steps:

- Generic syntax
- Generic interpreters

Running examples

Lecture focus on $\mathbb{Z}$ and $\mathbb{M}$
Exercises explore other examples similarly

Formal syntax

Finitary algebraic signature $\mathbf{S} = (\mathsf{Sort}_{\mathbf{S}}, \mathsf{Op}_{\mathbf{S}}, \mathsf{Arity}_{\mathbf{S}})$ (NB: indexed containers [Altenkirch et al. 14])

- **sort** set: $\mathsf{Sort}_{\mathbf{S}} \ni s$, written as decorated stars, e.g.: $\star_s$
- **operator** set: $\mathsf{Op}_{\mathbf{S}} \ni f$
- **arity** assignment $\mathsf{Arity}_{\mathbf{S}} : \mathsf{Op}_{\mathbf{S}} \to (\mathsf{List}\,\mathsf{Sort}_{\mathbf{S}}) \times \mathsf{Sort}_{\mathbf{S}}$

Instead of $\mathsf{Arity}_{\mathbf{S}} f = ([s_1, \ldots, s_n], r)$ write $(f : \star_{s_1} \times \cdots \times \star_{s_n} \to \star_r) \in \mathbf{S}$

Example: magma

- $\mathsf{Sort}_{\mathsf{Magma}} := \{_\}$ (i.e.: underline)
- $\mathsf{Op}_{\mathsf{Magma}} := \{(*), 1\}$
- $\mathsf{Arity}_{\mathsf{Magma}}(*) := ([_,_], _)$ so $((*) : \underline{\star} \times \underline{\star} \to \underline{\star}) \in \mathsf{Magma}$
- $\mathsf{Arity}\,1 = ([], _)$ so $1 : \underline{\star}$

Succinctly: $\mathsf{Magma} = \{(*) : \underline{\star} \times \underline{\star} \to \underline{\star}, 1 : \underline{\star}\}$

Example: reflexive quivers over set X

- $\mathsf{Sort}_{X\text{-}\mathsf{Quiver}} := \{\star_{m \times k} \mid m, k \in X\}$
- X-Quiver :=
  $\{(\cdot_{m,k,n}) : \star_{m\times k} \times \star_{k\times n} \to \star_{m\times n} \mid m, k, n \in X\}$
  $\cup \{\mathrm{I}_n : \star_{n\times n} \mid n \in X\}$

Interpretation

Algebra $\mathsf{A} = \left(\left(\mathsf{A}_s \right)_{s \in \mathsf{Sort}}, \left(\mathsf{f}_\mathsf{A} \right)_{\mathsf{f} \in \mathsf{Op}} \right)$ for signature $\mathbf{S}$

- Sort-indexed family of **carrier** sets: A_s for every $s \in \mathsf{Sort}$.
- Op-indexed family of **operations** between the corresponding carriers: $f_\mathsf{A} : \mathsf{A}_{s_1} \times \cdots \times \mathsf{A}_{s_n} \to \mathsf{A}_r$ for every operator $(f : \star_{s_1} \times \cdots \times \star_{s_n} \to \star_r) \in \mathbf{S}$

Example: magmas

algebras A for the signature

$\mathsf{Magma} = \left\{ (*) : \underline{\star} \times \underline{\star} \to \underline{\star}, 1 : \underline{\star} \right\}$

amount to:

- carrier set $\underline{\mathsf{A}}$
- operation $(*_\mathsf{A}) : \underline{\mathsf{A}}^2 \to \underline{\mathsf{A}}$
- constant $1_\mathsf{A} \in \underline{\mathsf{A}}$

Example: quivers

sets of matrices as algebra $\mathbb{M}$ for

$\mathbb{N}\text{-Quiver} := \left\{ (\cdot_{m,k,n}) : \star_{m \times k} \times \star_{k \times n} \to \star_{m \times n} \middle| m, k, n \in \mathbb{N} \right\} \cup \left\{ \mathrm{I}_n \middle| n \in \mathbb{N} \right\}$

- carriers: $\mathbb{M}_{m \times n}$ (m rows, n columns)
- multiplication: $\left(a_{i,j}\right)_{i,j} \cdot \left(b_{j,\ell}\right)_{j,\ell} := \sum_j a_{i,j} \cdot b_{j,\ell}$
- identity matrix: $\mathrm{I}_n := \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & & \vdots \\ \vdots & & \ddots & 0 \\ 0 & \cdots & 0 & 1 \end{pmatrix}$

Algebraic expressions

Algebras give **meta-level** expressions:

Example: magma $\mathsf{A} = \left(\underline{\mathsf{A}}, (\cdot) : \underline{\mathsf{A}}^2 \to \underline{\mathsf{A}}, 0\right)$

$$(x \cdot y) \cdot z \qquad x \cdot (y \cdot z) \qquad y \cdot (1 \cdot x) \qquad \left(x, y, z \in \underline{\mathsf{A}}\right)$$

Example: $\mathbb{N}$-Quiver of matrices $\mathbb{M}$

$$x \cdot (y \cdot z) \qquad x \cdot (y \cdot z) \qquad x \cdot (\mathrm{I}_k \cdot y) \qquad \left(x \in \mathbb{M}_{n\times k}, y \in \mathbb{M}_{k\times \ell}, z \in \mathbb{M}_{\ell\times p}\right)$$

Summary

We formulated the template and its instances:

- **Signatures** specify the operators and their argument and return **sorts**
- **Algebras** interpret these operators as operations of appropriate type

Both let us talk about our examples, and many other examples, generically.

Typically, consider sub-class of algebras **validating** axioms (e.g., **associativity**).
We cover these **varieties** later.

Course structure

Goal

relate **normalisation** and **modern algebra**

course URL

Zulip URL

- Part 1: universal algebra
  - Motivation
  - Universal algebra basics: signatures & algebras
  - Modern algebra: homomorphisms & representation theorems
  - Expression normalisation: free models & equational logic
  - Modern partial evaluation: free extensions
  - Categorical algebra basics
- Part 2: second-order multi-sorted algebra
- ~~Part 3: second-order normalisation~~

Modern algebra

The hallmarks of modern algebra are:

- studying **varieties** of algebras defined through **equational axioms**; and
- establishing **representation theorems** relating structures.

Both aspects pertinent for normalisation:

- The same **signature** can express many varieties of algebras.
  It is the variety that determines **normalisation**.
- The same **family** may carry multiple algebraic structures.
  The variety determines normalisation of **residual programs**.
- Representation theorems expose structural **properties**.

Structure-preserving-maps—**homomorphism**—are core to these relationship.

Structure-preserving maps

Homomorphism $h : \mathsf{A} \to \mathsf{B}$

between $\mathbf{S}$-algebras A, B:

- indexed family of functions $h_s : \mathsf{A}_s \to \mathsf{B}_s$, for all $s \in \mathsf{Sort}$;
- such that:

$$h_r\left(f_{\mathsf{A}}(a_1, \ldots, a_n)\right) = f_{\mathsf{B}}(h_{s_1} a_1, \ldots, h_{s_n} a_n)$$

for all $(f : \star_{s_1} \times \cdots \times \star_{s_n} \to \star_r) \in \mathbf{S}$ and $a_i \in \mathsf{A}_{s_i}$, for $i = 1, \ldots, n$

Examples

- Exponentiation is a magma homomorphism: $(\lambda x.2^x) : (\mathbb{Z}, (+), 0) \to (\mathbb{Q}, (\cdot), 1)$
- Uniform base-change is a quiver homomorphism:

$$\left(\lambda \mathsf{T} : \mathbb{M}_{m \times n}.\boldsymbol{J}_m \cdot \mathsf{T} \cdot \boldsymbol{J}_n^{-1}\right)_{m,n} : \mathbb{M} \to \mathbb{M}$$

for family of invertible matrices $\left(\boldsymbol{J}_n\right)_{n \in \mathbb{N}}$. (exercise: generalise)

Abstract syntax

Abstract syntax provides **object-level** terms, e.g., for first-class axioms
It'll be our first representation theorem:

$\mathsf{Term}_{\mathbf{S}} X$

over signature $\mathbf{S}$ and Sort-indexed family variables X— Sort-indexed inductive family:

$$\frac{}{x \in (\mathsf{Term}_{\mathbf{S}} X)_s}(x \in X_s) \qquad \frac{t_i \in (\mathsf{Term}_{\mathbf{S}} X)_{s_i} \text{ for all } i = 1, \ldots, n}{f(t_1, \ldots, t_n) \in (\mathsf{Term}_{\mathbf{S}} X)_r}((f : \star_{s_1} \times \cdots \times \star_{s_n} \to \star_r) \in \mathbf{S})$$

Write $X \vdash_{\mathbf{S}} t : \star_s$ when $t \in (\mathsf{Term}_{\mathbf{S}} X)_s$, and $(x : s) \in X$ when $x \in X_s$.
Data structure representing **abstract syntax trees** with $\mathbf{S}$-operators and X-variables.

Abstract syntax as an abstract data type

The operations available on the abstract syntax are:

- form a **variable**;
- form a composite **term** using an operator and appropriately sorted sub-terms;
- eliminate a term by **traversal**, given:
  - Motive $\underline{\mathsf{A}}_s$ for every sort;
  - Variable case: $e : X \to \underline{\mathsf{A}}$
  - Visitor for every AST node: $f_{\mathsf{A}} : \underline{\mathsf{A}}_{s_1} \times \cdots \times \underline{\mathsf{A}}_{s_n} \to \underline{\mathsf{A}}_r$

$h := \mathsf{fold}\left(\underline{\mathsf{A}}, e, \left(\mathsf{f}_{\mathsf{A}}\right)_{\mathsf{f}}\right) : \mathsf{Term}_{\mathbf{S}} X \to \underline{\mathsf{A}}$
uniquely defined by:

$h(\mathsf{var}\, x) = e_s x \qquad h(f(t_1, \ldots, t_n)) = f_{\mathsf{A}}(h\, t_1, \ldots, h\, t_n)$

Example

$\mathbf{S} := \mathsf{Magma} \quad \underline{X} := \{\mathsf{x}, \mathsf{y}, \mathsf{z}\} \quad \mathsf{A} := (\mathbb{Z}, (+), 0)$

$e : X = \{\mathsf{x}, \mathsf{y}, \mathsf{z}\} \to \mathbb{Z}$

$\mathsf{x} \mapsto 10 \quad \mathsf{y} \mapsto 5 \quad \mathsf{z} \mapsto -6$

$$\mathsf{fold}\,\mathsf{A}\, e\, ((\mathsf{y} \cdot \mathsf{x}) \cdot (1 \cdot \mathsf{z})) = (\overbrace{e\,\mathsf{y} + e\,\mathsf{x}}^{5+10}) + (\overbrace{0 + e\,\mathsf{z}}^{0+(-6)})$$
$$= 9$$

Abstract syntax as an abstract data type

The operations available on the abstract syntax are:

- form a **variable**;
  I.e., a family of functions $\mathsf{var} : X \to \mathsf{Term}_{\mathbf{S}} X$
- form a composite **term** using an operator and appropriately sorted sub-terms;
  I.e., $\mathbf{S}$-algebra structure $\mathsf{F}_{\mathbf{S}} X$:

$$(\mathsf{F}_{\mathbf{S}} X)_s := (\mathsf{Term}_{\mathbf{S}} X)_s \qquad f_{\mathsf{F}_{\mathbf{S}} X} : (\mathsf{F}_{\mathbf{S}} X)_{s_1} \times \cdots \times (\mathsf{F}_{\mathbf{S}} X)_{s_n} \to (\mathsf{F}_{\mathbf{S}} X)_r$$

- eliminate a term by **traversal**, given:
  - Motive $\underline{\mathsf{A}}_s$ for every sort;
  - Variable case: $e : X \to \underline{\mathsf{A}}$
  - Visitor for every AST node: $f_{\mathsf{A}} : \underline{\mathsf{A}}_{s_1} \times \cdots \times \underline{\mathsf{A}}_{s_n} \to \underline{\mathsf{A}}_r$

  $h := \mathsf{fold}\left(\underline{\mathsf{A}}, e, \left(\mathsf{f}_{\mathsf{A}}\right)_{\mathsf{f}}\right) : \mathsf{Term}_{\mathbf{S}} X \to \underline{\mathsf{A}}$
  uniquely defined by:
  $h(\mathsf{var}\, x) = e_s x \qquad h(f_{\mathsf{F}_{\mathbf{S}} X}(t_1, \ldots, t_n)) = f_{\mathsf{A}}(h\, t_1, \ldots, h\, t_n)$
  - i.e., each $\mathbf{S}$-algebra $\mathsf{A} = \left(\underline{\mathsf{A}}, \left(f_{\mathsf{A}}\right)\right)$;
  - and Sort-family map $e : X \to \underline{\mathsf{A}}$

  uniquely define:
  - **homomorphism** $\mathsf{fold}\,\mathsf{A}\,e : \mathsf{F}_{\mathbf{S}} X \to \mathsf{A}$
  - s.t. $\mathsf{fold}\,\mathsf{A}$ **extends** $e : X \to \underline{\mathsf{A}}$ along $\mathsf{var} : X \to \underline{\mathsf{F}_{\mathbf{S}} X}$.

Abstract syntax as a mathematical structure

Focus on the underlying maths:

- a family of functions $\mathsf{var} : X \to \mathsf{Term}_{\mathbf{S}} X$
- $\mathbf{S}$-algebra structure $\mathsf{F}_{\mathbf{S}} X$:
  $(\mathsf{F}_{\mathbf{S}} X)_s := (\mathsf{Term}_{\mathbf{S}} X)_s \qquad f_{\mathsf{F}_{\mathbf{S}} X} : (\mathsf{F}_{\mathbf{S}} X)_{s_1} \times \cdots \times (\mathsf{F}_{\mathbf{S}} X)_{s_n} \to (\mathsf{F}_{\mathbf{S}} X)_r$
- uniquely define:
  - **homomorphism** $\mathsf{fold}\,\mathsf{A}\,e : \mathsf{F}_{\mathbf{S}} X \to \mathsf{A}$
  - s.t. $\mathsf{fold}\,\mathsf{A}$ **extends** $e : X \to \underline{\mathsf{A}}$ along $\mathsf{var} : X \to \underline{\mathsf{F}_{\mathbf{S}} X}$.

  - i.e., each $\mathbf{S}$-algebra $\mathsf{A} = \left(\underline{\mathsf{A}}, \left(f_{\mathsf{A}}\right)\right)$;
  - and Sort-family map $e : X \to \underline{\mathsf{A}}$

Abstract syntax as a mathematical structure

Focus on the maths, rearrange into the situation:

- $\mathsf{F}_{\mathbf{S}}X$ is an $\mathbf{S}$-algebra:
$$(\mathsf{F}_{\mathbf{S}}X)_s \coloneqq (\mathsf{Term}_{\mathbf{S}}X)_s \quad f_{\mathsf{F}_{\mathbf{S}}X} : (\mathsf{F}_{\mathbf{S}}X)_{s_1} \times \cdots \times (\mathsf{F}_{\mathbf{S}}X)_{s_n} \to (\mathsf{F}_{\mathbf{S}}X)_r$$
- together with a family of functions $\mathsf{var} : X \to \mathsf{Term}_{\mathbf{S}}X$
- For every:
  - $\mathbf{S}$-algebra $\mathsf{A} = \left(\underline{\mathsf{A}}, \left(f_{\mathsf{A}}\right)\right)$
  - Sort-family map $e : X \to \underline{\mathsf{A}}$

  uniquely defines:
  - a **homomorphism** $\mathsf{fold}\,\mathsf{A}\,e : \mathsf{F}_{\mathbf{S}}X \to \mathsf{A}$
  - s.t. $\mathsf{fold}\,\mathsf{A}$ **extends** $e : X \to \underline{\mathsf{A}}$ along $\mathsf{var} : X \to \underline{\mathsf{F}_{\mathbf{S}}X}$.

Underlying abstraction:
algebra over X:

- $\mathbf{S}$-algebra $\mathsf{B} = \left(\underline{\mathsf{B}}, \left(f_{\mathsf{B}}\right)\right)$
- Sort-family map $d : X \to \underline{\mathsf{B}}$, the **environment/valuation**

and their **maps** $h : (\mathsf{B}, d) \to (\mathsf{A}, e)$:

- homomorphism $h : \mathsf{B} \to \mathsf{A}$
- preserving the environments:
  h extends e along d: $\quad e = h \circ d$

Abstract syntax as a mathematical structure

Focus on the maths, use underlying abstraction:

- $(\mathsf{F}_{\mathbf{S}}X, \mathsf{var})$ is an $\mathbf{S}$-algebra over X:

$$(\mathsf{F}_{\mathbf{S}}X)_s \coloneqq (\mathsf{Term}_{\mathbf{S}}X)_s \qquad f_{\mathsf{F}_{\mathbf{S}}X} : (\mathsf{F}_{\mathbf{S}}X)_{s_1} \times \cdots \times (\mathsf{F}_{\mathbf{S}}X)_{s_n} \to (\mathsf{F}_{\mathbf{S}}X)_r$$
$$\mathsf{var} : X \to \mathsf{Term}_{\mathbf{S}}X$$

- For every (A, e) $\mathbf{S}$-algebra over X, we have a unique map:

$$\mathsf{fold}\,\mathsf{A}\,e : \left(\mathsf{F}_{\mathbf{S}}X, \mathsf{var}\right) \to (\mathsf{A}, e)$$

Underlying abstraction:
algebra over X:

- $\mathbf{S}$-algebra $\mathsf{B} = \left(\underline{\mathsf{B}}, \left(f_{\mathsf{B}}\right)\right)$
- Sort-family map $d : X \to \underline{\mathsf{B}}$, the **environment/valuation**

and their **maps** $h : (\mathsf{B}, d) \to (\mathsf{A}, e)$:

- homomorphism $h : \mathsf{B} \to \mathsf{A}$
- preserving the environments:
  h extends e along d: $\quad e = h \circ d$

Representation theorem for abstract syntax

Our first **representation theorem**:

Theorem (abstract syntax initiality)

Let $\mathbf{S}$ *be a signature, and* X *a* $\mathsf{Sort}_{\mathbf{S}}$*-family. Then:*

- *abstract syntax trees for a signature* $\mathbf{S}$ *over a* $\mathsf{Sort}_{\mathbf{S}}$*-family of variables* X*;*
- *its* $\mathbf{S}$*-algebra structure* $\mathsf{F}_{\mathbf{S}}X$*:*

$$(\mathsf{F}_{\mathbf{S}}X)_s \coloneqq (\mathsf{Term}_{\mathbf{S}}X)_s \quad f_{\mathsf{F}_{\mathbf{S}}X} : (\mathsf{F}_{\mathbf{S}}X)_{s_1} \times \cdots \times (\mathsf{F}_{\mathbf{S}}X)_{s_n} \to (\mathsf{F}_{\mathbf{S}}X)_r$$

- *the variable map* $\mathsf{var} : X \to \underline{\mathsf{F}_{\mathbf{S}}X}$

form a **free $\mathbf{S}$-algebra over X***, i.e. an initial* $\mathbf{S}$*-algebra over* X*:*
for every $\mathbf{S}$*-algebra* (A, e) *over* X *there is a unique map* $\mathsf{fold}\,\mathsf{A}\,e : \left(\mathsf{F}_{\mathbf{S}}X, \mathsf{var}\right) \to (\mathsf{A}, e)$.

Succinctly—A-natural bijection:

$$\frac{e : X \longrightarrow \underline{\mathsf{A}}}{\mathsf{fold}\,\mathsf{A}\,e : \mathsf{F}_{\mathbf{S}}X \longrightarrow \mathsf{A}}$$

recover universal arrow as mate of identity:

$$\frac{\mathsf{var} : X \longrightarrow \underline{\mathsf{F}_{\mathbf{S}}X} = \mathsf{Term}_{\mathbf{S}}X}{\mathrm{id} : \mathsf{F}_{\mathbf{S}}X \longrightarrow \mathsf{F}_{\mathbf{S}}X}$$

Algebra and Normalisation: Lecture 2

Ohad Kammar

slides



course homepage



Zulip channel #Algebra and normalisation



Scottish Programming Languages and Verification Summer School (SPLV)
3–7 August 2026
University of Glasgow

THE UNIVERSITY of EDINBURGH informatics ifcs | Laboratory for Foundations of Computer Science | BayesCentre | Funded by: THE ROYAL SOCIETY

Recap: universal algebra basics

Universal algebra

We formulated the template and its instances:

- **Signatures** specify the operators and their argument and return **sorts**

$$\text{E.g.: } \mathsf{Magma} := \left\{(\cdot) : \underline{\star} \times \underline{\star} \to \underline{\star}, 1 : \underline{\star}\right\}$$

$$\text{Generally: } \mathbf{S} = \left(\mathsf{Sort} \in \mathbf{Set}, \mathsf{Op} \in \mathbf{Set}, \mathsf{Arity} : \mathsf{Op} \to \mathsf{List}\,\mathsf{Sort} \times \mathsf{Sort}\right)$$

- **Algebras** interpret these operators as operations of appropriate type:

$$\text{E.g., } \mathsf{Magma}\text{-algebra: } \mathsf{A} = \left(\underline{\mathsf{A}}, (\cdot) : \underline{\mathsf{A}}^2 \to \underline{\mathsf{A}}, 1 : \underline{\mathsf{A}}\right)$$

$$\text{Generally: } \mathsf{A} = \left(\left(\mathsf{A}_s\right)_{s \in \mathsf{Sort}} \in \mathbf{Fam}\,\mathsf{Sort}, \left(\mathsf{f}_{\mathsf{A}} : \mathsf{A}_{s_1} \times \cdots \times \mathsf{A}_{s_n} \to \mathsf{A}_r\right)_{(\mathsf{f} : \star_{s_1} \times \cdots \times \star_{s_n} \to \mathsf{r}) \in \mathbf{S}}\right)$$

Both let us talk about our examples, and many other examples, generically.

Abstract syntax

Abstract syntax provides **object-level** terms, e.g., for first-class axioms
It'll be our first representation theorem:

$\mathsf{Term}_{\mathbf{S}} X$

over signature $\mathbf{S}$ and Sort-indexed family variables X— Sort-indexed inductive family:

$$\frac{}{x \in (\mathsf{Term}_{\mathbf{S}} X)_s}(x \in X_s) \qquad \frac{t_i \in (\mathsf{Term}_{\mathbf{S}} X)_{s_i} \text{ for all } i = 1, \ldots, n}{f(t_1, \ldots, t_n) \in (\mathsf{Term}_{\mathbf{S}} X)_r}((f : \star_{s_1} \times \cdots \times \star_{s_n} \to \star_r) \in \mathbf{S})$$

Write $X \vdash_{\mathbf{S}} t : \star_s$ when $t \in (\mathsf{Term}_{\mathbf{S}} X)_s$, and $(x : s) \in X$ when $x \in X_s$.
Data structure representing **abstract syntax trees** with $\mathbf{S}$-operators and X-variables.

Representation theorem for abstract syntax

Our first **representation theorem**:

Theorem (abstract syntax initiality)

Let $\mathbf{S}$ *be a signature, and* X *a* $\mathsf{Sort}_{\mathbf{S}}$*-family. Then:*

- *abstract syntax trees for a signature* $\mathbf{S}$ *over a* $\mathsf{Sort}_{\mathbf{S}}$*-family of variables* X*;*
- *its* $\mathbf{S}$*-algebra structure* $\mathsf{F}_{\mathbf{S}}X$*:*

$$(\mathsf{F}_{\mathbf{S}}X)_s \coloneqq (\mathsf{Term}_{\mathbf{S}}X)_s \qquad f_{\mathsf{F}_{\mathbf{S}}X} : (\mathsf{F}_{\mathbf{S}}X)_{s_1} \times \cdots \times (\mathsf{F}_{\mathbf{S}}X)_{s_n} \to (\mathsf{F}_{\mathbf{S}}X)_r$$

- *the variable map* $\mathsf{var} : X \to \underline{\mathsf{F}_{\mathbf{S}}X}$

form a **free $\mathbf{S}$-algebra over X***, i.e. an initial* $\mathbf{S}$*-algebra over* X*:*
for every $\mathbf{S}$*-algebra* (A, e) *over* X *there is a unique map* $\mathsf{fold}\,\mathsf{A}\,e : \left(\mathsf{F}_{\mathbf{S}}X, \mathsf{var}\right) \to (\mathsf{A}, e)$.

Succinctly—A-natural bijection:

$$\frac{e : X \longrightarrow \underline{\mathsf{A}}}{\mathsf{fold}\,\mathsf{A}\,e : \mathsf{F}_{\mathbf{S}}X \longrightarrow \mathsf{A}}$$

recover universal arrow as mate of identity:

$$\frac{\mathsf{var} : X \longrightarrow \underline{\mathsf{F}_{\mathbf{S}}X} = \mathsf{Term}_{\mathbf{S}}X}{\mathrm{id} : \mathsf{F}_{\mathbf{S}}X \longrightarrow \mathsf{F}_{\mathbf{S}}X}$$

Representation theorems for abstract syntax

- The representation theorem isolates the **interface** to abstract syntax
- $(\mathsf{F}_{\mathbf{S}}X)_s \coloneqq (\mathsf{Term}_{\mathbf{S}}X)_s$ is one implementation

Another implementation: "concrete" syntax

Given coproduct diagram:

$$\mathsf{Op}_{\mathbf{S}} \xrightarrow{\mathsf{Op}} \mathsf{Symbol} \xleftarrow{\mathsf{Var}} X$$

Define $(\mathsf{C}_{\mathbf{S}}X)_s \subseteq \mathsf{List}\,\mathsf{Symbol}$:

$$\frac{}{[\mathsf{Var}x] \in (\mathsf{C}_{\mathbf{S}}X)_s}(x \in X_s) \qquad \frac{a_i \in (\mathsf{C}_{\mathbf{S}}X)_{s_i} \text{ for all } i = 1, \ldots, n}{(\mathsf{Op}f :: a_1 +\!\!+ \cdots +\!\!+ a_n) \in (\mathsf{C}_{\mathbf{S}}X)_r}((f : \star_{s_1} \times \cdots \times \star_{s_n} \to \star_r) \in \mathbf{S})$$

equip it with $\mathbf{S}$-algebra structure over X:

$$(\mathsf{G}_{\mathbf{S}}X)_s \coloneqq (\mathsf{C}_{\mathbf{S}}X)_s \qquad f_{\mathsf{G}_{\mathbf{S}}X}(a_1, \ldots, a_n) \coloneqq (\mathsf{Op}f :: a_1 +\!\!+ \cdots +\!\!+ a_n) \qquad \mathsf{var}_s x \coloneqq [\mathsf{Var}x]$$

Representation theorems for abstract syntax

Theorem ("concrete" syntax initiality)

Let $\mathbf{S}$ *be a signature, and* X *a* $\mathsf{Sort}_{\mathbf{S}}$*-family. Take* Symbol, $\mathsf{C}_{\mathbf{S}}X$ *from previous slide. The following* $\left(\mathsf{G}_{\mathbf{S}}X, \mathsf{var}\right)$*:*

$$(\mathsf{G}_{\mathbf{S}}X)_s \coloneqq (\mathsf{C}_{\mathbf{S}}X)_s \qquad f_{\mathsf{G}_{\mathbf{S}}X}(a_1, \ldots, a_n) \coloneqq (\mathsf{Op}f \mathbin{::} a_1 +\!\!+ \cdots +\!\!+ a_n) \qquad \mathsf{var}_s x \coloneqq [\mathsf{Var}x]$$

is a free $\mathbf{S}$*-algebra over* X*.*

(Exercise sheet explores this implementation and others in more detail.)

Course structure

Goal

relate **normalisation** and **modern algebra**

- Part 1: universal algebra
  - Motivation
  - Universal algebra basics: signatures & algebras
  - Modern algebra: homomorphisms & representation theorems
  - Expression normalisation: free models & equational logic
  - Modern partial evaluation: free extensions
  - Categorical algebra basics
- Part 2: second-order multi-sorted algebra
- ~~Part 3: second-order normalisation~~



course URL



Zulip URL

Main takeaway

Core idea: algebra and normalisation

representation theorem provides **normalisation** procedure

Already appears in **abstract syntax**—different concrete representations

"(x_ + _y)" vs. "$x + y$"

6 characters 3 characters

reflected by same abstract representation:

$$x + y \qquad \text{i.e.:} \begin{cases} \text{tree representation:} & (+)(x, y) \\ \text{“concrete” representation:} & \left[\mathsf{Op}\,(+), \mathsf{Var}\,x, \mathsf{Var}\,y\right] \end{cases}$$

reifying the representation as a string gives the **same** concrete syntax, say:

"$x + y$"

We'll unpack this for **monoids**.

Monoid representation

Monoid $\mathsf{A} = \left(\underline{\mathsf{A}}, (\cdot) : \underline{\mathsf{A}}^2 \to \underline{\mathsf{A}}, 1 : \mathsf{A}^2\right)$

associative unital magma:

- **associative:**

$$(x \cdot y) \cdot z = x \cdot (y \cdot z) \qquad (x, y, z \in \underline{\mathsf{A}})$$

- **unital:**

$$x \cdot 1 = x = 1 \cdot x \qquad (x \in \underline{\mathsf{A}})$$

monoid homomorphism $h : \mathsf{A} \to \mathsf{B}$ is a magma homomorphism

Monoid A over set X

- function $d : X \to \underline{\mathsf{A}}$, the **environment**

and their maps $h : (\mathsf{A}, d) \to (\mathsf{B}, e)$:

- homomorphism $h : \mathsf{A} \to \mathsf{B}$
- preserving the environment: h extends e along d.

Monoid representation

Theorem

Let X be a set. The following data defines a free monoid over X:

$$\underline{\mathsf{F}_{\mathsf{Monoid}} X} \coloneqq \mathsf{List}\, X \qquad u \cdot v \coloneqq u +\!\!+ v \qquad 1 \coloneqq [] \qquad \eta^{\mathsf{Monoid}} x \coloneqq [x]$$

I.e.:

- $\mathsf{F}_{\mathsf{Monoid}} X \coloneqq (\mathsf{List}\, X, (+\!\!+), [])$ *is a monoid, and*
- *for every $e : X \to \underline{\mathsf{A}}$ monoid A over X there is a unique monoid homomorphism:*

$$\mathsf{eval}\, \mathsf{A}\, e : \mathsf{F}_{\mathsf{Monoid}} X \to \mathsf{A}$$

extending e along the **unit** η^{Monoid}*. It is given by:*

$$\mathsf{eval}\, \mathsf{A}\, e\, [x_1, \ldots, x_n] \coloneqq 1 \cdot (e\, x_1) \cdot \cdots \cdot (e\, x_n)$$

$$\frac{e : X \longrightarrow \underline{\mathsf{A}} \text{ in } \mathbf{Set}}{\mathsf{eval}\, \mathsf{A}\, e : \mathsf{F}_{\mathsf{Monoid}} X \longrightarrow \mathsf{A} \text{ in } \mathbf{Monoid}}$$

Monoid normalisation

Core idea: algebra and normalisation

representation theorem provides **normalisation** procedure

We have a representation theorem, so we deserve a normalisation procedure:

Theorem (extensional normalisation, informally)

Let $\left(\mathsf{F}_{\mathsf{Monoid}} X, \eta^{\mathsf{Monoid}}\right)$ *be a free monoid over the set* X*, and* t, s *be monoid expressions with variables from* X*.*
The equation $t = u$ *is valid in all monoids iff it is valid in* $\mathsf{F}_{\mathsf{Monoid}} X$*.*

To phrase and prove this theorem precisely, we need to define **validity** of equations.

Validity

Equation $X \vdash_{\mathbf{S}} t = u : \star_s$ over signature $\mathbf{S}$

- set X
- terms $t, s \in (\mathsf{Term}_{\mathbf{S}} X)_s$

Satisfaction $(\mathsf{A}, e) \vDash (X \vdash t = u : \star_s)$

by $\mathbf{S}$-algebra $e : X \to \underline{\mathsf{A}}$ over X:

$$\mathsf{fold}\,\mathsf{A}\,e\,t = \mathsf{fold}\,\mathsf{A}\,e\,u$$

Algebra A **validates** equation $\mathsf{A} \vDash (X \vdash t = u : \star_s)$ when every environment satisfies it:

$$\forall e : X \to \underline{\mathsf{A}}. \qquad (\mathsf{A}, e) \vDash (X \vdash t = u : \star_s)$$

Validity

Satisfaction $(\mathsf{A}, e) \vDash (X \vdash t = u : \star_s)$

by $\mathbf{S}$-algebra $e : X \to \underline{\mathsf{A}}$ over X:

$$\mathsf{fold}\,\mathsf{A}\,e\,t = \mathsf{fold}\,\mathsf{A}\,e\,u$$

Algebra A **validates** equation $\mathsf{A} \vDash (X \vdash t = u : \star_s)$ when every environment satisfies it:

$$\forall e : X \to \underline{\mathsf{A}}. \qquad (\mathsf{A}, e) \vDash (X \vdash t = u : \star_s)$$

Examples

- The unit of the list monoid satisfies the equation:

  $$(\mathsf{F}_{\mathsf{Monoid}}\,\{x, y\}\,, \eta^{\mathsf{Monoid}}) \vDash (\{x, y\} \vdash (x \cdot 1) \cdot y = 1 \cdot (x \cdot y) : \underline{\star}$$

  since:

  $$([x] +\!\!+ []) +\!\!+ [y] = [x, y] = [] +\!\!+ ([x] +\!\!+ [y])$$

Validity

Satisfaction $(\mathsf{A}, e) \vDash (X \vdash t = u : \star_s)$

by $\mathbf{S}$-algebra $e : X \to \underline{\mathsf{A}}$ over X:

$$\mathsf{fold}\,\mathsf{A}\,e\,t = \mathsf{fold}\,\mathsf{A}\,e\,u$$

Algebra A **validates** equation $\mathsf{A} \vDash (X \vdash t = u : \star_s)$ when every environment satisfies it:

$$\forall e : X \to \underline{\mathsf{A}}. \qquad (\mathsf{A}, e) \vDash (X \vdash t = u : \star_s)$$

Examples

- In fact, every monoid validates the equation:

$$\mathsf{A} \vDash (\{x, y\} \vdash (x \cdot 1) \cdot y = 1 \cdot (x \cdot y) : \underline{\star})$$

since:

$$\begin{array}{ccccc} & & \overset{\text{unitality}}{\downarrow} & & \overset{\text{unitality}}{\downarrow} \\ ((e\,x) \cdot 1) \cdot (e\,y) & = & (e\,x) \cdot (e\,y) & = & 1 \cdot ((e\,x) \cdot (e\,y)) \end{array}$$

for every environment $e : \{x, y\} \to \underline{\mathsf{A}}$

Validity

Satisfaction $(\mathsf{A}, e) \vDash (X \vdash t = u : \star_s)$

by $\mathbf{S}$-algebra $e : X \to \underline{\mathsf{A}}$ over X:

$$\mathsf{fold}\,\mathsf{A}\,e\,t = \mathsf{fold}\,\mathsf{A}\,e\,u$$

Algebra A **validates** equation $\mathsf{A} \vDash (X \vdash t = u : \star_s)$ when every environment satisfies it:

$$\forall e : X \to \underline{\mathsf{A}}. \qquad (\mathsf{A}, e) \vDash (X \vdash t = u : \star_s)$$

Examples

- A monoid is a Magma-algebra A validating the associativity and unitality:

$$\mathsf{A} \vDash (\{x, y, z\} \vdash (x \cdot y) \cdot z = x \cdot (y \cdot z) : \underline{\star}) \qquad \mathsf{A} \vDash (\{x\} \vdash x \cdot 1 = x = 1 \cdot x : \underline{\star})$$

Validity

Satisfaction $(\mathsf{A}, e) \vDash (X \vdash t = u : \star_s)$

by $\mathbf{S}$-algebra $e : X \to \underline{\mathsf{A}}$ over X:

$$\mathsf{fold}\,\mathsf{A}\,e\,t = \mathsf{fold}\,\mathsf{A}\,e\,u$$

Algebra A **validates** equation $\mathsf{A} \vDash (X \vdash t = u : \star_s)$ when every environment satisfies it:

$$\forall e : X \to \underline{\mathsf{A}}. \qquad (\mathsf{A}, e) \vDash (X \vdash t = u : \star_s)$$

Lemma (homomorphic images preserve equations)

Let $h : \mathsf{A} \to \mathsf{B}$ be an $\mathbf{S}$-homomorphism. Then:[1]

$$(\mathsf{A}, e) \vDash (X \vdash t = u : \star_s) \implies (\mathsf{B}, h \circ e) \vDash (X \vdash t = u : \star_s)$$

So if $h : \underline{\mathsf{A}} \twoheadrightarrow \underline{\mathsf{B}}$ is **surjective**, *then: $\mathsf{A} \vDash (X \vdash t = u : \star_s) \implies \mathsf{B} \vDash (X \vdash t = u : \star_s)$.*

[1] The slides given in the lecture had a mistake here, thanks Orpheas van Rooij for the correction.

Monoid normalisation

Theorem (extensional normalisation)

Let $\left(\mathsf{F}_{\mathsf{Monoid}}X, \eta^{\mathsf{Monoid}}\right)$ *be a free monoid over* X*, and* $X \vdash t = u : \underline{\star}$ *an equation. The following are equivalent (TFAE):*

1. *The unit satisfies the equation:* $(\mathsf{F}_{\mathsf{Monoid}}X, \eta^{\mathsf{Monoid}}) \vDash (X \vdash t = u : \underline{\star})$
2. *The free monoid validates the equation:* $\mathsf{F}_{\mathsf{Monoid}}X \vDash (X \vdash t = u : \underline{\star})$
3. *Every monoid* $\mathsf{A} \in \mathbf{Monoid}$ *validates the equation:* $\mathsf{A} \vDash (X \vdash t = u : \underline{\star})$

Proof idea, implemented over next few slides

By suitable instantiation: (1) $\Longleftarrow$ (2) $\Longleftarrow$ (3). For (3) $\Longleftarrow$ (1):

- Develop proof theory for univeral algebra: **equational logic**
- By **soundness** of equational logic, a provable equation is valid in all monoids
- The **quotient** of the free magma by **provability** is a **free monoid**. ■

Proving this theorem for monoids is as easy as proving it for every variety of algebras.

Varieties of algebras

Presentation $\mathcal{P} = (\mathbf{S}, \mathbf{Ax})$

- signature $\mathbf{S}$
- set $\mathbf{Ax}$ of $\mathbf{S}$-equations $(X \vdash t = u) \in \mathbf{Ax}$, the **axioms**

$\mathcal{P}$-**model**: $\mathbf{S}$-algebra A validating every axiom $\mathsf{A} \vDash (X \vdash t = u : \star_s) \in \mathbf{Ax}$
The class of models for some presentation is called a **variety** of algebras.

Example: monoids

Monoid $= \left(\mathsf{Magma}, \mathbf{Ax}_{\mathsf{Monoid}}\right)$ presentation:

$$\{x, y, z\} \vdash (x \cdot y) \cdot z = x \cdot (y \cdot z) : \underline{\star} \qquad \{x\} \vdash x \cdot 1 = x = 1 \cdot x : \underline{\star}$$

monoids are Monoid-models

Quotienting an algebra

Congruence ($\equiv$) over $\mathbf{S}$-algebra A

Sort-indexed family of equivalence relations ($\equiv_s$) over A_s that respects the operations:

$$\frac{a \in \mathsf{A}_s}{a \equiv_s a}(\mathsf{refl}) \qquad \frac{a \equiv_s b}{b \equiv_s a}(\mathsf{sym}) \qquad \frac{a \equiv_s b \qquad b \equiv_s c}{a \equiv_s c}(\mathsf{trans})$$

$$\frac{X \vdash e_1 \equiv e_2 \qquad X \vdash t : \star_s}{\mathsf{fold}\,\mathsf{A}\,e_1\,t \equiv_s \mathsf{fold}\,\mathsf{A}\,e_2\,t}(\mathsf{cong})$$

where $X \vdash e_1 \equiv e_2$ for environments $e_1, e_2 : X \to \underline{\mathsf{A}}$ when:

$$e_1\,x \equiv_s e_2\,x \text{ for every } (x : \star_s) \in X.$$

Example: length parity over $\mathsf{F}_{\mathsf{Monoid}} X = (\mathsf{List}\,X, (+\!\!+), [])$.

$u \equiv_{\underline{\star}} v$ when u,v both odd-length or both even-length is a congruence

Quotienting an algebra

Quotient $\mathbf{S}$-algebra $\mathsf{A}/(\equiv)$

(where $(\equiv)$ congruence over A)

$$(\mathsf{A}/(\equiv))_s := \mathsf{A}_s/(\equiv_s) \qquad f_{\mathsf{A}/(\equiv)}([a_1]_{s_1}, \ldots, [a_n]_{s_n}) := [f_{\mathsf{A}}(a_1, \ldots, a_n)]_r$$
$$((f : \star_{s_1} \times \cdots \times \star_{s_n} \to s_r) \in \mathbf{S})$$

Lemma

The quotient map is a $\mathbf{S}$*-homomorphism* $[-]_ : \mathsf{A} \to \mathsf{A}/(\equiv)$*, it is the universal* $(\equiv)$**-respecting homomorphism** *out of* A*:*

- *for all* $a \equiv_s b$*, we have* $[a]_s = [b]_s$*;*
- *for every* $h : \mathsf{A} \to \mathsf{B}$ *satisfying* $a \equiv_s b \implies h_s a = h_s b$*, there is a unique homomorphism* $\therefore h : \mathsf{A}/(\equiv) \to \mathsf{B}$ *extending* h *along* $[-]$*.*

Equational logic: preliminaries

A substitution $X \vdash_{\mathbf{S}} \theta : Y$

family map $\mathsf{Term}_{\mathbf{S}} X \xleftarrow{\theta} Y$, i.e.: $(X \vdash \theta\, x : \star_s) \mapsto (x : \star_s) \in Y$

Substitution operation (given $X \vdash_{\mathbf{S}} \theta : Y$)

family map $- [\theta] : \mathsf{Term}_{\mathbf{S}} X \xleftarrow{\mathsf{fold}(\mathsf{F}_{\mathbf{S}} X)\, \theta} \mathsf{Term}_{\mathbf{S}} Y$, i.e.:

$$(X \vdash t\,[\theta] : \star_s) \mapsto (Y \vdash t : \star_s)$$

Example

Substitution does what you think it should:

$$((x \cdot y) \cdot (x \cdot 1))\overbrace{[x \mapsto z, y \mapsto 1 \cdot z]}^{\theta :=} = ((\theta x) \cdot (\theta y)) \cdot ((\theta x) \cdot 1) = (z \cdot (1 \cdot z)) \cdot (z \cdot 1)$$

Equational logic

Provability $X \vdash_{\mathcal{P}} t = u : \star_s$

$\mathbf{S}$-congruence closure of the axiom:

$$\frac{}{X \vdash_{\mathcal{P}} t = t : \star_s}(\text{refl}) \qquad \frac{X \vdash_{\mathcal{P}} t = u : \star_s}{X \vdash_{\mathcal{P}} u = t : \star_s}(\text{sym})$$

$$\frac{X \vdash_{\mathcal{P}} t = u : \star_s \qquad X \vdash_{\mathcal{P}} u = v : \star_s}{X \vdash_{\mathcal{P}} t = v : \star_s}(\text{trans})$$

$$\frac{X \vdash_{\mathcal{P}} \theta_1 = \theta_2 : Y \quad Y \vdash_{\mathbf{S}} t : \star_s}{X \vdash_{\mathcal{P}} t\left[\theta_1\right] = t\left[\theta_2\right] : \star_s}(\text{cong}) \qquad \frac{(Y \vdash t = u : \star_s) \in \mathbf{Ax}_{\mathcal{P}} \quad X \vdash_{\mathbf{S}} \theta : Y}{X \vdash_{\mathcal{P}} t[\theta] = u[\theta] : \star_s}(\text{axiom})$$

where $X \vdash_{\mathcal{P}} \theta_1 = \theta_2 : Y$ when $X \vdash \theta_1 x = \theta_2 x : \star_r$ for all $(x : \star_r) \in Y$.

Equational logic

Example

We write equational proof referring to axioms only:

$$\{x, y\} \vdash_{\mathsf{Monoid}} (x \cdot 1) \cdot y = x \cdot y = 1 \cdot (x \cdot y)$$

(unitality, unitality)

but mean derivations:

$$\dfrac{\dfrac{\dfrac{\dfrac{(\{x\} \vdash x \cdot 1 = x) \in \mathbf{Ax}}{\{x, y\} \vdash x \cdot 1 = x}(\text{axiom}) \qquad \dfrac{}{y = y}(\text{refl})}{\{x, y\} \vdash (z \mapsto x \cdot 1, y \mapsto y) = (z \mapsto x, y \mapsto y) : \{z, y\}}}{(x \cdot 1) \cdot y = x \cdot y}(\text{cong}) \qquad \dfrac{(z = 1 \cdot z) \in \mathbf{Ax}}{x \cdot y = 1 \cdot (x \cdot y)}(\text{axiom})}{\{x, y\} \vdash (x \cdot 1) \cdot y = 1 \cdot (x \cdot y) : \underline{\star}}(\text{trans})$$

Relating proofs and models

Theorem (soundness)

For every $\mathcal{P}$-model A:

$$X \vdash_{\mathcal{P}} t = u : \star_s \implies \mathsf{A} \vDash (X \vdash t = u : \star_s)$$

Proof

By induction on the derivation of the provability relation.

- **reflexivity, symmetry, transitivity** use corresponding property of equality, e.g.:

  $$(\mathsf{A}, e) \vDash t = u \implies \mathsf{fold}\,\mathsf{A}\,e\,t = \mathsf{fold}\,\mathsf{A}\,e\,u \implies \mathsf{fold}\,\mathsf{A}\,e\,u = \mathsf{fold}\,\mathsf{A}\,e\,t \implies (\mathsf{A}, e) \vDash u = t$$

- **congruence** uses Leibniz's law, and basic properties of substitutions:

  $$\underset{\text{substitution lemma}}{\mathsf{fold}\,\mathsf{A}\,e\,(t\,[\theta_1])} = \underset{\text{IH+Leibniz's law}}{\mathsf{fold}\,\mathsf{A}\,(\mathsf{fold}\,\mathsf{A}\,e \circ \theta_1)\,(t) = \mathsf{fold}\,\mathsf{A}\,(\mathsf{fold}\,\mathsf{A}\,e \circ \theta_2)\,(t)} = \mathsf{fold}\,\mathsf{A}\,e\,(t\,[\theta_2]) \implies (\mathsf{A}, e) \vDash t\,[\theta_1] = t\,[\theta_2]$$

- **axiom** uses the $\mathcal{P}$-model assumpion. ■

Free model by quotienting

Theorem (quotiented algebra representation)

Let $\mathcal{P}$ be a presentation and X a family. We have a free $\mathcal{P}$-model over X:

$$\mathsf{F}_{\mathcal{P}}X \coloneqq \mathsf{F}_{\mathbf{S}}X/(X \vdash (=)) \qquad \eta^{\mathcal{P}} : X \xrightarrow{\mathsf{var}} \mathsf{F}_{\mathbf{S}}X \xrightarrow{[-]} \big(\mathsf{F}_{\mathbf{S}}X/(=)\big) =: \underline{\mathsf{F}_{\mathcal{P}}X}$$

I.e.,for every $\mathcal{P}$-model A and family map $e : X \to \underline{\mathsf{A}}$ there is a unique $\mathbf{S}$-homomorphism extending e along $\eta^{\mathcal{P}}$ given by $\therefore \mathsf{fold}\,\mathsf{A}\,e : \mathsf{F}_{\mathcal{P}}X \to \mathsf{A}$.

Proof assembles existing parts:

$$\frac{e : X \longrightarrow \underline{\mathsf{A}} \text{ family map}}{\mathsf{fold}\,\mathsf{A}\,e : \mathsf{F}_{\mathbf{S}}X \longrightarrow \mathsf{A} \ \ \mathbf{S}\text{-homomorphism, by freeness}}$$

$$\therefore \mathsf{fold}\,\mathsf{A}\,e : \mathsf{F}_{\mathcal{P}}X \coloneqq \mathsf{F}_{\mathbf{S}}X/(X \vdash (=)) \longrightarrow \mathsf{A} \ \ \mathbf{S}\text{-homomorphism, by lemma}$$

■

Normalisation via representability

Theorem (extensional normalisation)

Let $(\mathsf{F}_{\mathcal{P}}X, \eta^{\mathcal{P}})$ *be a free* $\mathcal{P}$*-model over a family* X*, and* $X \vdash t = u : \underline{\star}$ *an equation. The following are equivalent (TFAE):*

1. *The unit satisfies the equation:* $(\mathsf{F}_{\mathcal{P}}X, \eta^{\mathcal{P}}) \vDash (X \vdash t = u : \underline{\star})$
2. *The given free* $\mathcal{P}$*-model validates the equation:* $\mathsf{F}_{\mathcal{P}}X \vDash (X \vdash t = u : \underline{\star})$
3. *Every* $\mathcal{P}$*-model* A *validates the equation:* $\mathsf{A} \vDash (X \vdash t = u : \underline{\star})$
4. *The equation is provable from* $\mathcal{P}$*:* $X \vdash_{\mathcal{P}} t = u : \underline{\star}$

Proof

Suitable instantiation/soundness: (1) $\Longleftarrow$ (2) $\Longleftarrow$ (3) $\Longleftarrow$ (4). For (4) $\Longleftarrow$ (1):

- The free $\mathcal{P}$-models are canonically isomorphic $h : (\mathsf{F}_{\mathcal{P}}X, \eta^{\mathcal{P}}) \xrightarrow{\cong} (\mathsf{F}_{\mathbf{S}}X/(=), [\mathsf{var}])$.
- Homo. images preserve equations $\Longrightarrow (\mathsf{F}_{\mathbf{S}}X/(=), [\mathsf{var}]) \vDash t = u \Longrightarrow X \vdash_{\mathcal{P}} t = u$

as we wanted. ∎

Summary

Core idea: algebra and normalisation

representation theorem for an **effective** algebra	provides	extensional **normalisation** procedure, i.e.: **decide equality**

Refinement: **effective** algebra and normalisation

effective representation theorem for an **effective** algebra	provides	intensional **normalisation** procedure, i.e.: decide a derivation of every equation

- Prove a constructive representation theorem for the free **setoid** $\mathcal{P}$-model.
- Evaluate the equation in the effective free algebra, and transport it to the setoid-quotient free $\mathcal{P}$-model.

See *Frex: dependently-typed algebraic simplification*. Guillaume Allais, Edwin Brady, Nathan Corbyn, Ohad Kammar, and Jeremy Yallop. 2025. DOI: 10.1145/3747506.

Algebra and Normalisation: Lecture 3

Ohad Kammar

slides

course homepage

Zulip channel #Algebra and normalisation

Scottish Programming Languages and Verification Summer School (SPLV)
3–7 August 2026
University of Glasgow

THE UNIVERSITY of EDINBURGH informatics lfcs | Laboratory for Foundations of Computer Science | BayesCentre | Funded by: THE ROYAL SOCIETY

Recap: universal algebra

Presentation $\mathcal{P} = (\mathbf{S}, \mathbf{Ax})$

- signature $\mathbf{S}$
- set $\mathbf{Ax}$ of $\mathbf{S}$-equations $(X \vdash t = u) \in \mathbf{Ax}$, the **axioms**

$\mathcal{P}$-model: $\mathbf{S}$-algebra A validating every axiom $\mathsf{A} \vDash (X \vdash t = u : \star_s) \in \mathbf{Ax}$
The class of models for some presentation is called a **variety** of algebras.
Needed some technology from logic:

- **satisfaction:** $(\mathsf{A}, X \xrightarrow{e} \underline{\mathsf{A}}) \vDash (X \vdash t = u : \star_s)$ when: $\mathsf{fold}\,\mathsf{A}\,e\,t = \mathsf{fold}\,\mathsf{A}\,e\,u$.
- **validity:** $\mathsf{A} \vDash (X \vdash t = u : \star_s)$ when $\quad \forall e : X \to \underline{\mathsf{A}}. \quad (\mathsf{A}, e) \vDash (X \vdash t = u : \star_s)$

Example

monoids are Monoid-models, for the presentation $\mathsf{Monoid} = \left(\mathsf{Magma}, \mathbf{Ax}_{\mathsf{Monoid}}\right)$:

$$\{x, y, z\} \vdash (x \cdot y) \cdot z = x \cdot (y \cdot z) : \underline{\star} \qquad \{x\} \vdash x \cdot 1 = x = 1 \cdot x : \underline{\star}$$

Recap: equational logic

The **proof theory** of algebra, the provability relation $X \vdash_{\mathcal{P}} t = u : \star_s$ is the $\mathbf{S}$-congruence closure of the axioms:

$$\frac{}{X \vdash_{\mathcal{P}} t = t : \star_s}(\text{refl}) \qquad \frac{X \vdash_{\mathcal{P}} t = u : \star_s}{X \vdash_{\mathcal{P}} u = t : \star_s}(\text{sym})$$

$$\frac{X \vdash_{\mathcal{P}} t = u : \star_s \qquad X \vdash_{\mathcal{P}} u = v : \star_s}{X \vdash_{\mathcal{P}} t = v : \star_s}(\text{trans})$$

$$\frac{X \vdash_{\mathcal{P}} \theta_1 = \theta_2 : Y \quad Y \vdash_{\mathbf{S}} t : \star_s}{X \vdash_{\mathcal{P}} t\left[\theta_1\right] = t\left[\theta_2\right] : \star_s}(\text{cong}) \qquad \frac{(Y \vdash t = u : \star_s) \in \mathbf{Ax}_{\mathcal{P}} \quad X \vdash_{\mathbf{S}} \theta : Y}{X \vdash_{\mathcal{P}} t\,[\theta] = u\,[\theta] : \star_s}(\text{axiom})$$

where $X \vdash_{\mathcal{P}} \theta_1 = \theta_2 : Y$ when $X \vdash \theta_1 x = \theta_2 x : \star_r$ for all $(x : \star_r) \in Y$.

Recap: meta-theory

Theorem (soundness)

For every $\mathcal{P}$-model A:

$$X \vdash_{\mathcal{P}} t = u : \star_s \implies \mathsf{A} \vDash (X \vdash t = u : \star_s)$$

Theorem (quotiented algebra representation)

Let $\mathcal{P}$ be a presentation and X a family. We have a free $\mathcal{P}$-model over X:

$$\mathsf{F}_{\mathcal{P}} X \coloneqq \mathsf{F}_{\mathbf{S}} X / (X \vdash (=)) \qquad \eta^{\mathcal{P}} : X \xrightarrow{\text{var}} \mathsf{F}_{\mathbf{S}} X \xrightarrow{[-]} \left(\mathsf{F}_{\mathbf{S}} X / (=)\right) =: \underline{\mathsf{F}_{\mathcal{P}} X}$$

I.e.,for every $\mathcal{P}$-model A and family map $e : X \to \underline{\mathsf{A}}$ there is a unique $\mathbf{S}$-homomorphism extending e along $\eta^{\mathcal{P}}$.

This theorem does not help us at all to normalise expressions directly.
Satisfying $[\theta] \vDash (X \vdash t = u)$ in $\mathsf{F}_{\mathbf{S}} Y / (Y \vdash (=))$ amounts to proving $Y \vdash t\,[\theta] = u\,[\theta]$.

Core idea: representation and normalisation

A representation theorem for an **effective** free $\mathcal{P}$-model over X, does help:

Theorem (extensional normalisation)

Let $(\mathsf{F}_{\mathcal{P}}X, \eta^{\mathcal{P}})$ be a free $\mathcal{P}$-model over a family X, and $X \vdash t = u : \underline{\star}$ an equation. The following are equivalent (TFAE):

1. *The unit satisfies the equation:* $(\mathsf{F}_{\mathcal{P}}X, \eta^{\mathcal{P}}) \vDash (X \vdash t = u : \underline{\star})$
2. *The given free $\mathcal{P}$-model validates the equation:* $\mathsf{F}_{\mathcal{P}}X \vDash (X \vdash t = u : \underline{\star})$
3. *Every $\mathcal{P}$-model A validates the equation:* $\mathsf{A} \vDash (X \vdash t = u : \underline{\star})$
4. *The equation is provable from $\mathcal{P}$:* $X \vdash_{\mathcal{P}} t = u : \underline{\star}$

$\eta^{\mathsf{Monoid}} := z \mapsto [z]$ makes the **effective** monoid $(\mathsf{List}\,\{x, y\}, (+\!\!+), [])$ **free** over $\{x, y\}$

To prove the equation $\{x, y\} \vdash_{\mathsf{Monoid}} (x \cdot 1) \cdot y = 1 \cdot (x \cdot y) : \underline{\star}$ evaluate with η^{Monoid}:

$([x]+\!\!+[])+\!\!+[y] = [x, y] = []+\!\!+([x]+\!\!+[y]) \quad \Longrightarrow \quad \eta^{\mathsf{Monoid}} \vDash (\{x, y\} \vdash (x \cdot 1) \cdot y = 1 \cdot (x \cdot y) : \underline{\star})$

Summary

Core idea: algebra and normalisation

representation theorem for an **effective** algebra	provides	extensional **normalisation** procedure, i.e.: **decide equality**

Refinement: **effective** algebra and normalisation

effective representation theorem for an **effective** algebra	provides	intensional **normalisation** procedure, i.e.: decide a derivation of every equation

- Prove a constructive representation theorem for the free **setoid** $\mathcal{P}$-model.
- Evaluate the equation in the effective free algebra, and transport it to the setoid-quotient free $\mathcal{P}$-model.

See *Frex: dependently-typed algebraic simplification*. Guillaume Allais, Edwin Brady, Nathan Corbyn, Ohad Kammar, and Jeremy Yallop. 2025. DOI: 10.1145/3747506.

Course structure

Goal

relate **normalisation** and **modern algebra**

course URL

Zulip URL

- Part 1: universal algebra
  - Motivation
  - Universal algebra basics: signatures & algebras
  - Modern algebra: homomorphisms & representation theorems
  - Expression normalisation: free models & equational logic
  - Modern partial evaluation: free extensions
  - Categorical algebra basics
- Part 2: second-order multi-sorted algebra
- ~~Part 3: second-order normalisation~~

Modern partial evaluation: motivation

Partially-static expressions

mixing:

- **constants**;
- algebraic operators; and
- variables

E.g., over the monoid $(\mathbb{Q}, (\cdot), 0)$:

$$-\frac{3}{4}+x+\frac{1}{4}+y+\frac{1}{2} = x+y$$

Variables can represent:

- function arguments;
- expression holes;
- delayed-evaluation place-holders; or
- sub-expressions outside the signature $\mathbf{S}$ (e.g., uninterpreted function calls)

Partially-static representations

Extension (E, s) of $\mathcal{P}$-model A

- $\mathcal{P}$-model E;
- $\mathcal{P}$-homomorphism $s : \mathsf{A} \to \mathsf{E}$;

Extension homomorphism $h : (\mathsf{E}, s) \to (\mathsf{D}, r)$:

- $\mathcal{P}$-model homomorphism $h : \mathsf{E} \to \mathsf{D}$;
- extending $r : \mathsf{A} \to \mathsf{D}$ along $s : \mathsf{A} \to \mathsf{E}$.

I.e., the **coslice category** $\mathsf{A}/\mathbf{Mod}\,\mathcal{P}$

Freely extending monoids

Theorem (monoid frex representation)

Let $\mathsf{A} = \left(\underline{\mathsf{A}}, (\cdot), 1\right)$ *be a* **monoid**, *and* X *a set.*
The **free extension** (**frex**) *of* A *by* X *is given by* **alternating lists**:

$$\underline{\mathsf{A}[X]} \coloneqq \left\{ \left[a_0, x_1, a_1, \ldots, x_n, a_n\right] \middle| n \in \mathbb{N}, a_i \in \underline{\mathsf{A}}, x_i \in X \right\} \qquad 0_{\mathsf{A}[X]} \coloneqq \left[0_{\mathsf{A}}\right]$$

$$\left[a_0, x_1, a_1, \ldots, x_n, a_n\right] \cdot_{\mathsf{A}[X]} \left[b_0, y_1, b_1, \ldots, y_k, b_k\right]$$

$$\coloneqq \left[a_0, x_1, a_1, \ldots, x_n, a_n \cdot b_0, y_1, b_1, \ldots, y_k, b_k\right]$$

$$s : \mathsf{A} \to \mathsf{A}[X] \quad s\, a \coloneqq [a] \qquad \eta : X \to \underline{\mathsf{A}[X]} \quad \eta\, x \coloneqq \left[0_{\mathsf{A}}, x, 0_{\mathsf{A}}\right]$$

I.e., for every extension $r : \mathsf{A} \to \mathsf{E}$ *and environment map* $e : X \to \underline{\mathsf{E}}$, *there is a unique extension homomorphism* $[r, e] : (\mathsf{A}[X], s) \to (\mathsf{E}, r)$ *extending* e *along* η, *given by:*

$$[r, e]\left[a_0, x_1, a_1, \ldots, x_n, a_n\right] \coloneqq (r\, a_0) \cdot_{\mathsf{E}} (e\, x_1) \cdot_{\mathsf{E}} (r\, a_1) \cdot_{\mathsf{E}} \cdots \cdot_{\mathsf{E}} (e\, x_n) \cdot_{\mathsf{E}} (r\, a_n)$$

Freely extending commutative monoids

Adding commutativity $\{x, y\} \vdash x{+}y = y{+}x : \underline{\star}$ simplifies the representation:

Theorem (commutative monoid frex representation)

Let; $\mathsf{A} = \left(\underline{\mathsf{A}}, (+), 0\right)$ *be a* **commutative monoid**, X *a set,* $\mathcal{M}X$ *multisets over* X. *The* **free extension** *(***frex***) of* A *by* X *is given by* **pairing an** A**-constant with a bag/multiset of** X*:*

$$\underline{\mathsf{A}\,[X]} \coloneqq \underline{\mathsf{A}} \times \mathcal{M}X \qquad 0_{\mathsf{A}[X]} \coloneqq \left(0_{\mathsf{A}}, []\right) \qquad (a,u) +_{\mathsf{A}[X]} (b,v) \coloneqq (a{+}b, u \cup v)$$

$$s : \mathsf{A} \to \mathsf{A}\,[X] \qquad s\,a \coloneqq (a, \wr\!\int) \qquad \eta : X \to \underline{\mathsf{A}\,[X]} \qquad \eta\,x \coloneqq \wr x \int$$

I.e., for every extension $r : \mathsf{A} \to \mathsf{E}$ *and environment map* $e : X \to \underline{\mathsf{E}}$*, there is a unique extension homomorphism* $[r, e] : (\mathsf{A}\,[X], s) \to (\mathsf{E}, r)$ *extending* e *along* η*, given by:*

$$[r, e]\,(a, u) \coloneqq (r\,a) +_{\mathsf{E}} \sum_{x \in u} e\,x$$

NB, we sum by multiplicity, e.g.: $[r, e]\,(a, \wr x, x, y \int) \coloneqq (r\,a) +_{\mathsf{E}} (e\,x) +_{\mathsf{E}} (e\,x) +_{\mathsf{E}} (e\,y)$

Presenting extensions

Free extensions of $\mathcal{P}$-models reduce to free models:

Evaluation presentation $\mathsf{Eval}\,\mathsf{A}$ in $\mathbf{S}$-algebra A

- Add constants for each A-element: $\mathbf{S}_{\mathsf{Eval}\,\mathsf{A}} := \mathbf{S} \cup \left\{ \underline{a} : \star_s \middle| a \in \mathsf{A}_s \right\}$
- Evaluation equation for each operator:

$$\mathbf{Ax}_{\mathsf{Eval}\,\mathsf{A}} := \left\{ \begin{array}{l} \{x_1, \ldots, x_n\} \vdash \\ f(\underline{a}_1, \ldots, \underline{a}_n) = \underline{f_{\mathsf{A}}(a_1, \ldots, a_n)} : \star_r \end{array} \middle| \begin{array}{l} (f : \star_{s_1} \times \cdots \times \star_{s_n} \to \star_r) \in \mathbf{S}, \\ a_i \in \mathsf{A}_{s_i} \end{array} \right\}$$

For example:

$$\left(\underline{\frac{1}{2}} + - \underline{\frac{1}{4}} \right) = \underline{\frac{1}{4}} \in \mathbf{Ax}_{\mathsf{Eval}(\mathbb{Q},+,0)}$$

For $\mathcal{P}$-model A, the **A-extension presentation** $\mathcal{P}_{\mathsf{Eval}\,\mathsf{A}}$:

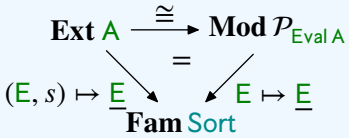
$$\mathbf{S}_{\mathcal{P}_{\mathsf{Eval}\,\mathsf{A}}} := \mathbf{S}_{\mathsf{Eval}\,\mathsf{A}} \qquad \mathbf{Ax}_{\mathcal{P}_{\mathsf{Eval}\,\mathsf{A}}} := \mathbf{Ax}_{\mathcal{P}} \cup \mathbf{Ax}_{\mathsf{Eval}\,\mathsf{A}}$$

Presenting extensions

There is nothing more to the theory of frex than our current development:

Theorem (extensions as models)

Let A *be a* $\mathcal{P}$*-model.*
The categories of A*-extensions and* $\mathcal{P}_{\mathsf{Eval}\,\mathsf{A}}$*-models are isomorphic:*



so the frex of A *by* X *is the free* $\mathcal{P}_{\mathsf{Eval}\,\mathsf{A}}$*-model over* X*.*

This perspective sometimes help in constructing the frex.

Course structure

Goal

relate **normalisation** and **modern algebra**

- Part 1: universal algebra
  - Motivation
  - Universal algebra basics: signatures & algebras
  - Modern algebra: homomorphisms & representation theorems
  - Expression normalisation: free models & equational logic
  - Modern partial evaluation: free extensions
  - Categorical algebra basics
- Part 2: second-order multi-sorted algebra
- ~~Part 3: second-order normalisation~~



course URL



Zulip URL

Categorical reformulation

In Part 2:

programs involving **object-level variables** and **substitution**

These incur 'noise'. **Abstract** from noise using **category theory**

categorical algebra dictionary

signature $\mathbf{S}$	$\mapsto$	functor $\mathbf{S} : \mathbf{Fam}\,\mathsf{Sort} \to \mathbf{Fam}\,\mathsf{Sort}$.
algebra $\mathsf{A} = \left(\left(\mathsf{A}_s\right)_s, \left(\mathsf{f}_\mathsf{A}\right)_\mathsf{f}\right)$ for $\mathbf{S}$	$\mapsto$	algebra $\mathsf{A} = \left(\underline{\mathsf{A}}, \mathbf{S}\underline{\mathsf{A}} \xrightarrow{(-)_\mathsf{A}} \underline{\mathsf{A}}\right)$ for the functor $\mathbf{S}$
variety of $\mathcal{P}$-models	$\mapsto$	category $\mathcal{A}$
free $\mathcal{P}$-model $\mathsf{F}_\mathcal{P}$ construction variable embedding $\eta : X \to \underline{\mathsf{F}_\mathcal{P}}$ and homomorphic evaluation: $\mathsf{eval}(X \to \underline{\mathsf{A}}) : \mathsf{F}_\mathcal{P} X \to \mathsf{A}$ terms and substitution	$\mapsto$	adjunction $\mathsf{F} \dashv (U : \mathcal{A} \to \mathbf{Fam}\,\mathsf{Sort})$ monad/Kleisli triple $T = \left(U \circ \mathsf{F}, \eta, \gg\!=\right)$

From **signatures** and algebras to **functors** and their algebras, modularly

Idea #1

Each operator with **arity** $f : \star_{s_1} \times \cdots \times \star_{s_n} \to \star_r$ yields **nodes** in the AST

- the sort of resulting **node** r
- the sort of each **sub-structure** s_i

The family $\mathbf{S}_f X$ of f-nodes with sub-structures given by $X \in \mathbf{Fam}\,\mathsf{Sort}$ is:

$$(\mathbf{S}_f X)_{r'} := \begin{cases} r' = r : & X_{s_1} \times \cdots \times X_{s_n} \\ \text{otherwise} : & \emptyset \end{cases}$$

Example: Magma and $\mathbb{N}$-Quiver

$\underline{\mathsf{Magma}_{(\cdot)} X := \underline{X} \times \underline{X}}$

$$(\mathbb{N}\text{-}\mathsf{Quiver}_{(\cdot)_{(m,k,n)}} X)_{m \times n} := X^{m \times k} \times X^{k \times n} \qquad (\mathbb{N}\text{-}\mathsf{Quiver}_{(\cdot)_{(m,k,n)}} X)_{m' \times n'} := \emptyset$$

Idea #2

An algebra A yields a visitor for f-nodes in the AST, folding A-results for substructures into results for the node:

$f_{\mathsf{A}} : \mathbf{S}_f \underline{\mathsf{A}} \to \mathsf{A} \qquad (f_{\mathsf{A}})_r(a_1, \ldots, a_n) \coloneqq f_{\mathsf{A}}(a_1, \ldots, a_n) \qquad (f_{\mathsf{A}})_{r'}$ is the empty function

Example: magmas and matrices

$\underline{(\cdot)_{\mathsf{A}}} : (a_1, a_2) \mapsto a_1 \cdot a_2 \qquad (\cdot_{(m,k,n),\mathbb{M}})_{m \times n} : (M \in \mathbb{M}_{m \times k}, K \in \mathbb{M}_{k \times n}) \mapsto M \cdot K \in \mathbb{M}_{m \times n}$

Idea #3

The homomorphism condition:

$$h_r(f_{\mathsf{A}}(a_1, \ldots, a_n)) = f_{\mathsf{B}}(h_{s_1}\, a_1, \ldots, h_{s_n}\, a_n)$$

uses the **functorial action**:

$$(\mathbf{S}_f(X \xrightarrow{h} Y))_r : (\mathbf{S}_f X)_r \coloneqq X_{s_1} \times \cdots \times X_{s_n} \ni (x_1, \ldots, x_n) \mapsto (h\, x_1, \ldots, h\, x_n) \in (\mathbf{S}_f Y)_r$$

which we express as:

$$\begin{array}{ccc} (\mathbf{S}_f \underline{\mathsf{A}})_r \ni (a_1, \ldots, a_n) & \xmapsto{\quad (\mathbf{S}_f h)_r \quad} & (h_{s_1}\, a_1, \ldots, h_{s_1}\, a_n) \in (\mathbf{S}_f \underline{\mathsf{B}})_r \\ \Big\downarrow f_{\mathsf{A},r} & & \Big\downarrow f_{\mathsf{B},r} \\ \underline{\mathsf{A}}_r \ni f_{\mathsf{A}}(a_1, \ldots, a_n) & \xmapsto[h_r]{} & h_r(f_{\mathsf{A}}(a_1, \ldots, a_n)) = f_{\mathsf{B}}(h\, a_1, \ldots, h\, a_n) \in \underline{\mathsf{B}}_r \end{array}$$

Idea #3

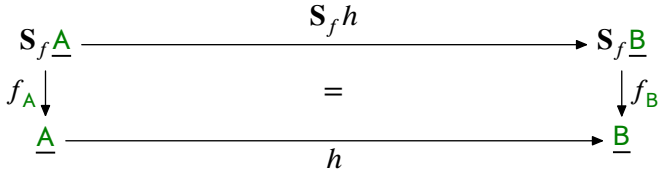
The homomorphism condition:

$$h_r(f_{\mathsf{A}}(a_1, \ldots, a_n)) = f_{\mathsf{B}}(h_{s_1}\, a_1, \ldots, h_{s_n}\, a_n)$$

uses the **functorial action**:

$$(\mathbf{S}_f(X \xrightarrow{h} Y))_r : (\mathbf{S}_f X)_r \coloneqq X_{s_1} \times \cdots \times X_{s_n} \ni (x_1, \ldots, x_n) \mapsto (h\, x_1, \ldots, h\, x_n) \in (\mathbf{S}_f Y)_r$$

which we express as:



Idea #4

These ideas combine using coproducts:

- Signature functor:

$$\mathbf{S}X \coloneqq (\mathsf{f} : \mathsf{Op}_{\mathbf{S}}) \times \mathbf{S}_{\mathsf{f}}$$

- Algebras A for the signature amount to functions out of the functor at the carrier:

$$\left(\underline{\mathsf{A}}_{\mathsf{s}_1} \times \cdots \times \underline{\mathsf{A}}_{\mathsf{s}_n} \to \underline{\mathsf{A}}_{\mathsf{r}}\right)_{(\mathsf{f} : \star_{\mathsf{s}_1} \times \cdots \times \star_{\mathsf{s}_n} \to \star_{\mathsf{r}}) \in \mathsf{Op}} \quad \text{becomes:} \quad \mathbf{S}\underline{\mathsf{A}} \xrightarrow{(-)_{\mathsf{A}}} \underline{\mathsf{A}}$$

- The homomorphism condition for $h : \underline{\mathsf{A}} \to \underline{\mathsf{B}}$ amounts to the equation:

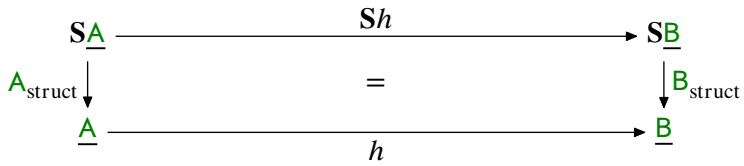
$$\begin{array}{ccc} \mathbf{S}\underline{\mathsf{A}} & \xrightarrow{\quad \mathbf{S}h \quad} & \mathbf{S}\underline{\mathsf{B}} \\ {\scriptstyle -_{\mathsf{A}}}\downarrow & = & \downarrow{\scriptstyle -_{\mathsf{B}}} \\ \underline{\mathsf{A}} & \xrightarrow[\quad h \quad]{} & \underline{\mathsf{B}} \end{array}$$

From **signatures** and algebras to **functors** and their algebras, modularly

Putting these ideas together:

Algebra $\mathsf{A} = \left(\underline{\mathsf{A}}, \mathsf{A}_{\text{struct}}\right)$ for functor $\mathbf{S} : \mathcal{C} \to \mathcal{C}$

morphism $\mathsf{A}_{\text{struct}} : \mathbf{S}\underline{\mathsf{A}} \to \underline{\mathsf{A}}$.
Homomorphism of algebras $h : \mathsf{A} \to \mathsf{B}$ is morphism $h : \underline{\mathsf{A}} \to \underline{\mathsf{B}}$ satisfying:



These concepts subsume the algebraic situation via:

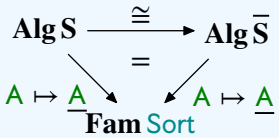
$$\mathbf{S}X \coloneqq (\mathsf{f} : \mathsf{Op}_{\mathbf{S}}) \times \mathbf{S}_{\mathsf{f}} \qquad \text{and} \qquad \mathsf{A}_{\text{struct}} \coloneqq -_{\mathsf{A}} \qquad \mathsf{B}_{\text{struct}} \coloneqq -_{\mathsf{B}}$$

Signature algebras are functor algebras

Theorem

Let $\mathbf{S}$ *be an algebraic signature and* $\overline{\mathbf{S}} : \mathbf{Fam}\,\mathsf{Sort} \to \mathbf{Fam}\,\mathsf{Sort}$ *its signature functor. The following two categories are isomorphic:*

- *algebras for the signature* $\mathbf{S}$ *and*
- *algebras for the functor* $\overline{\mathbf{S}}$



Abstract syntax abstractly

Thanks to this theorem and the representation theorems for abstract syntax, we generalise:

- Algebras over a family $X \in \mathcal{C}$ amount to the **comma category**

$$X \downarrow \underline{-} \qquad \text{objects: } (\mathsf{A} \in \mathbf{Alg}\,\mathbf{S}, X \xrightarrow{d} \underline{\mathsf{A}}) \qquad \begin{array}{c}\text{morphisms } (\mathsf{A}, d) \xrightarrow{h} (\mathsf{B}, e) : \\ \text{algebra homo. } h : \mathsf{A} \to \mathsf{B} \\ \text{extending } e \text{ along } d\end{array}$$

- Abstract syntax with variables from $X \in \mathcal{C}$ is the left-adjoint

$$\mathsf{F}_{\mathbf{S}} \dashv (\mathbf{Alg}\,\mathbf{S} \xrightarrow{\underline{-}} \mathcal{C})$$

- The resulting monad abstracts the object of terms:

$$\mathsf{Term}_{\mathbf{S}} X \coloneqq \underline{\mathsf{F}_{\mathbf{S}} X} \qquad [\mathsf{var}] \coloneqq \eta^{\mathbf{S}} : X \to \mathsf{Term}_{\mathbf{S}} X \qquad \mathsf{eval}\,\mathsf{A}\,(X \to \underline{\mathsf{A}}) : \mathsf{F}_{\mathbf{S}} X \to \mathsf{A}$$

$$-[X \xrightarrow{\theta} \mathsf{Term}_{\mathbf{S}} Y] \coloneqq \underline{\mathsf{eval}\,(\mathsf{F}_{\mathbf{S}} X)\,\eta^{\mathbf{S}}} : \mathsf{Term}_{\mathbf{S}} X \to \mathsf{Term}_{\mathbf{S}} Y$$

Varieties abstractly

Q: What role did the signature functor $\mathbf{S}$ play in the previous slide?
A: It doesn't appear there!

Varieties abstractly

- Start with any functor what-so-ever $\underline{_} : \mathcal{A} \to \mathcal{C}$.
  - objects in $\mathcal{C}$ are **abstract carriers**
  - objects in $\mathcal{A}$ are **abstract models**
  - morphisms in $\mathcal{A}$ are **abstract homomorphisms**
- Algebras over a family $X \in \mathcal{C}$ amount to the comma category

$$X \downarrow \underline{_}$$

- Abstract syntax with variables from $X \in \mathcal{C}$ is the left-adjoint

$$\mathsf{F} \dashv (\mathcal{A} \xrightarrow{\underline{_}} \mathcal{C}) \quad \text{with} \quad \left(\mathsf{F}X, \eta_X\right) \in \mathcal{A} \text{ initial}$$

- The resulting monad abstracts the object of terms:

$$\mathsf{Term}_{\mathbf{S}} X := \underline{\mathsf{F}_{\mathbf{S}} X} \qquad [\mathsf{var}] := \eta^{\mathbf{S}} : X \to \mathsf{Term}_{\mathbf{S}} X \qquad \mathsf{eval}\,\mathsf{A}\,(X \to \underline{\mathsf{A}}) : \mathsf{F}_{\mathbf{S}} X \to \mathsf{A}$$

$$-[X \xrightarrow{\theta} \mathsf{Term}_{\mathbf{S}} Y] := \underline{\mathsf{eval}\,(\mathsf{F}_{\mathbf{S}} X)\,\eta^{\mathbf{S}}} : \mathsf{Term}_{\mathbf{S}} X \to \mathsf{Term}_{\mathbf{S}} Y$$

Further topics in categorical algebra

- Q: Can we **recover** all $\mathcal{P}$-models from the free $\mathcal{P}$-model monad $T_{\mathcal{P}} \coloneqq \underline{\mathsf{F}_{\mathcal{P}}}$?
  A: Always for varieties, but not for general adjunctions.
  Read more about **monadic** adjunctions and **Beck's monadicity** theorem.
- Q: Which monads come from algebraic varieties?
  A: **finitary** monads, as extensions of **Lawvere theories**
- Q: We worked over set-theoretic categories ($\mathbf{Fam}\,\mathsf{Sort}$).
  Can we work over other categories?
  A: Answers galore.
  To work with **equations**, can define $\mathcal{P}$-models in a category with **finite products**.
  To **generalise** equations, many concepts bagged under **monad-theory** correspondence. Relevant buzzwords:
  - Relative monads
  - Monads with arities
  - Enriched Lawvere theories
  - Sound limit-doctrines, including parameterised algebraic theories

Course structure

Goal

relate **normalisation** and **modern algebra**

course URL

Zulip URL

- Part 1: universal algebra
  - Motivation
  - Universal algebra basics: signatures & algebras
  - Modern algebra: homomorphisms & representation theorems
  - Expression normalisation: free models & equational logic
  - Modern partial evaluation: free extensions
  - Categorical algebra basics
- Part 2: second-order multi-sorted algebra
- ~~Part 3: second-order normalisation~~

Course structure

Goal

relate **normalisation** and **modern algebra**

- Part 1: universal algebra
- Part 2: second-order multi-sorted algebra
  - Motivation
  - Categories of contexts: finite product completion
  - Object-level and meta-level variables
  - Axiomatising object-level substitution
  - Signatures & terms
  - Equations
- ~~Part 3: second-order normalisation~~



course URL



Zulip URL

Motivation: from expressions to programs

Goal

treat **program** normalisation algebraically

Main distinction for us:

Object-level identifiers in terms

$$(\lambda x : \mathbb{Z}.x + 1)(y - 3) \quad \text{vs} \quad -2 + y$$

Object-level substitution in terms and equations

$$\{\mathbf{m} : (x : A \vdash \mathsf{comp}\, B), \mathbf{v} : (\vdash \mathsf{val}\, A)\} \vdash$$

$$(\lambda x : A.\mathbf{m}\,[x])(\mathbf{v}) = \mathbf{m}\,[x \mapsto \mathbf{v}] : \mathsf{comp}\, B$$

Motivation: from expressions to programs

Goal

treat **program** normalisation algebraically

Main distinction for us:

Object-level identifiers in terms

$$(\lambda x : \mathbb{Z}.x + 1)(y - 3) \quad \text{vs} \quad -2 + y$$

Object-level substitution in terms and equations

(in e.g. Idris, we write second-order variables $\mathbf{m}$ as `?m` and call them **holes**)

first-order variable considered as a second-order variable

$$\{\underbrace{\mathbf{m} : (x : A \vdash \text{comp } B)}_{\text{second-order variable}}, \overbrace{\mathbf{v} : (\vdash \text{val } A)}\} \vdash \overbrace{(\lambda \underbrace{x : A}_{\text{object-level identifier}} .\mathbf{m}\,[x])(\mathbf{v}) = \mathbf{m}\,[\overbrace{x \mapsto \mathbf{v}}\,]}^{\text{object-level substitution}} : \text{comp } B$$

Universe for second-order algebra

In Part 1, we organised terms with variables in X into a **family**:

$$\mathsf{Term}_{\mathbf{S}} X \;:\; \mathsf{Sort} \to \mathbf{Set}$$

Today, Nachi taught us that terms and their semantics are organised into **presheaves**. Informally:

$$\mathsf{Term}_{\mathbf{S}} X \;:\; \mathsf{Sort} \times \mathbf{Ctx}^{\mathrm{op}} \to \mathbf{Set} \qquad \text{i.e.:} \qquad \mathsf{Term}_{\mathbf{S}} X \in \mathbf{PSh}(\mathsf{Sort} \times \mathbf{Ctx}_{\mathsf{Sort}})$$

To make this formal, we need to define:

- category $\mathbf{Ctx}_{\mathsf{Sort}}$
- what type of thing X is
- interface to object-level substitution
- what type of thing $\mathbf{S}$ is
- representation theorem for $\mathsf{Term}_{\mathbf{S}} X$

Course structure

Goal

relate **normalisation** and **modern algebra**

- Part 1: universal algebra
- Part 2: second-order multi-sorted algebra
  - Motivation
  - Categories of contexts: finite product completion
  - Object-level and meta-level variables
  - Axiomatising object-level substitution
  - Signatures & terms
  - Equations
- ~~Part 3: second-order normalisation~~



course URL



Zulip URL

Category of contexts

Our contexts will be **structural**, i.e., support Cartesian operations:

- embedding of bindable sorts $A \mapsto \mathsf{x} : A$;
- context **contatenation**;
- **permutation** $\Gamma, \Delta \to \Delta, \Gamma$;
- **weakening** $\Gamma, \Delta \to \Gamma$; and
- **duplication** $\Gamma \to \Gamma, \Gamma$

We use the **free finite-product completion** of the bindable sorts: $S \mapsto S_{\vdash}$, it's a **category** even when S is a set. Free constructions on categories are 2-**dimensional**.

Dimensions

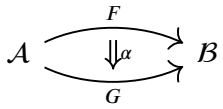
We'll need to move from 1-dimensional to 2-dimensional category theory:

- Sets are 0-**dimensional**: they only have elements
- Categories are 1-**dimensional**: they have objects and morphisms

To study **relationships** we typically need to move 1-dimension up.

Categories, functors, and natural transformations

Yesterday, Bob taught us the ingredients of 1-dimensional category theory:



- $\mathcal{A}$, $\mathcal{B}$ are **categories**, i.e.:
  - collections of 0-**cells**: the objects $A \in \mathcal{A}$ and in $\mathcal{B}$
  - collections of 1-**cells**: the morphisms $A \xrightarrow{f} B$ in A.

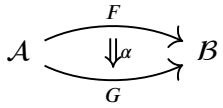
  with:
  - identities $A \xrightarrow{\text{id}} A$
  - composition $g \circ f : A \xrightarrow{f} B \xrightarrow{g} C$

  such that:

every commutative diagram is a meaningful statement

Categories, functors, and natural transformations

Yesterday, Bob taught us the ingredients of 1-dimensional category theory:



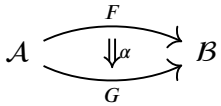
- $F, G : \mathcal{A} \to \mathcal{B}$ are **functors**, i.e.:
  - map sending 0-**cells** in $\mathcal{A}$ to 0-**cells** in $\mathcal{B}$: $\mathcal{A} \ni A \mapsto FA \in \mathcal{B}$
  - map sending 1-**cells** in $\mathcal{A}$ to 1-**cells** in $\mathcal{B}$: $(A \xrightarrow{f} B) \mapsto FA \xrightarrow{Ff} FB$
  - map sending commuting squares in $\mathcal{A}$ to commuting squares in $\mathcal{B}$:

$$\begin{array}{ccc} A & = & B \\ \| & & \| \\ A & & B \end{array} \quad \mapsto \quad \begin{array}{ccc} FA & = & FB \\ \| & & \| \\ FA & & FB \end{array}$$

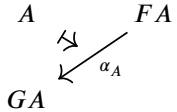
$$\begin{array}{ccc} & B & \\ {\scriptstyle f}\nearrow & = & \searrow{\scriptstyle g} \\ A & & C \\ & {\scriptstyle g\circ f}\searrow \quad \nearrow & \end{array} \quad \mapsto \quad \begin{array}{ccc} & FB & \\ {\scriptstyle Ff}\nearrow & = & \searrow{\scriptstyle Fg} \\ FA & & FC \\ & {\scriptstyle F(g\circ f)}\searrow \quad \nearrow & \end{array}$$

Categories, functors, and natural transformations

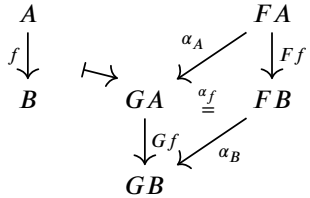
Yesterday, Bob taught us the ingredients of 1-dimensional category theory:



- $\alpha : F \Rightarrow G : \mathcal{A} \to \mathcal{B}$ is a **natural transformation**, i.e.:
  - map sending 0-**cells** in $\mathcal{A}$ to 1-**cells** in $\mathcal{B}$:



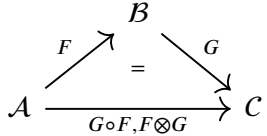
  - map sending 1-**cells** in $\mathcal{A}$ to 2-**cells** in $\mathcal{B}$:



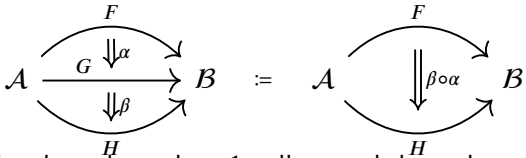
Pasting diagrams

2-dimensional category theory studies 0-,1,2-cells with identities and:

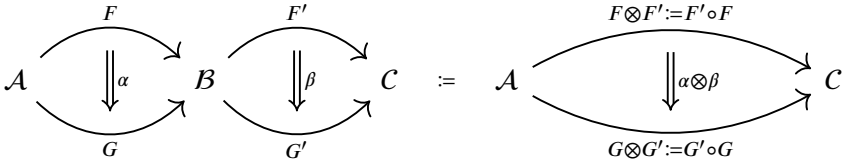
- **1-cell composition:** 1-cells whose boundary 0-cells match:
  And in the next two slides we'll define:



- **2-cell vertical composition:** 2-cells whose boundary 1-cells match:



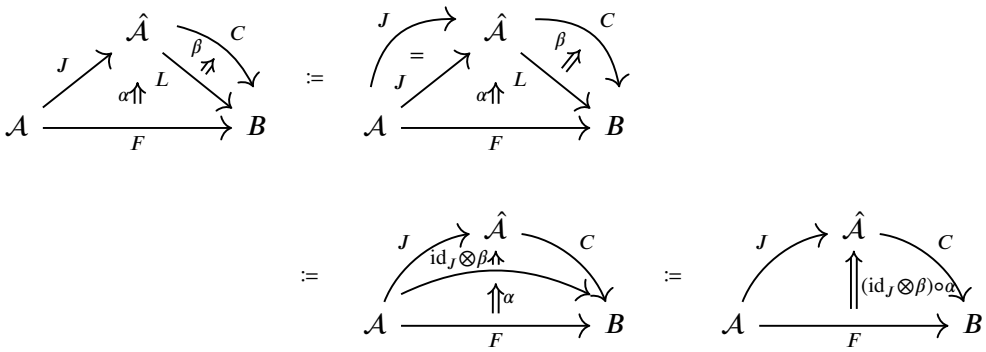
- **2-cell horizontal composition:** 2-cells whose boundary 1-cells match boundary:



such that: every pasting diagram is a meaningful 2-cell

Every pasting diagram is a meaningful 2-cell

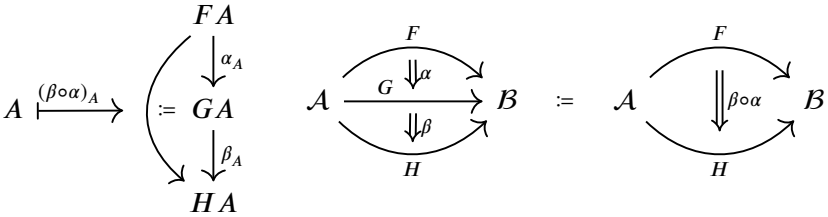
Example



The 2-category axioms (we won't cover those) ensure the resulting 2-cell does not depend on the order of 1-cell and 2-cell composition.

2-cell composition in **CAT**

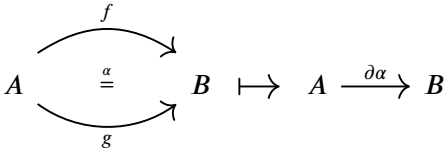
Vertical composition



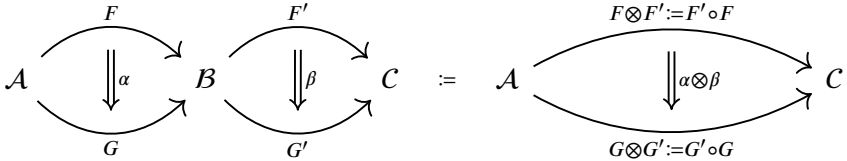
Every pasting diagram is a meaningful 2-cell

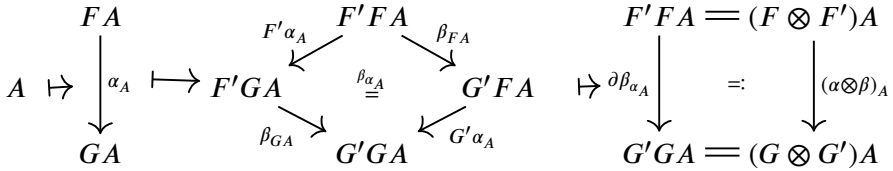
Observation: commuting diagrams

A commuting diagram equates its two sides, justifying the 'middle' 1-cell:



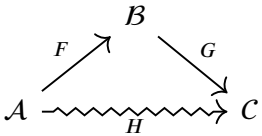
Horizontal composition



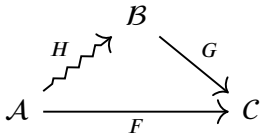


Types of 2-dimensional factorisation

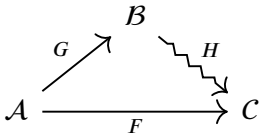
Factorising 1-cells (F, G inputs; H output)



H composes F before G

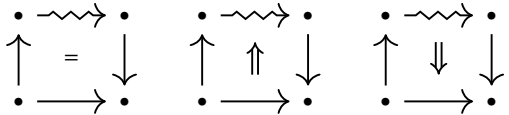


H lifts F along G



H extends F along G

Types of factorisation



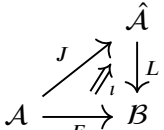
mere left right

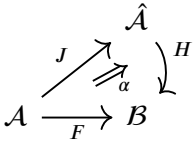
Kan

Universal

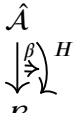
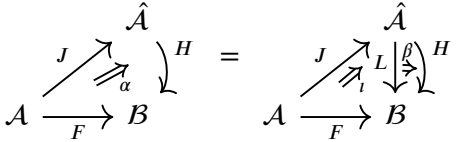
Types of 2-dimensional factorisation

Example: left Kan extension (L, ι) of F along J

a left 1-cell extension  such that:

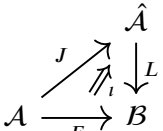
for all extensions of F along J 

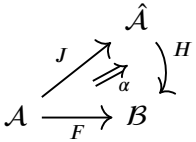
there exists

a unique 2-cell extension  of α along ι: 

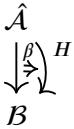
Types of 2-dimensional factorisation

Example: left Kan extension (L, ι) of F along J

a left 1-cell extension  such that:

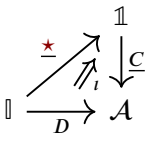
for all extensions of F along J 

there exists

a unique 2-cell extension  of α along ι:

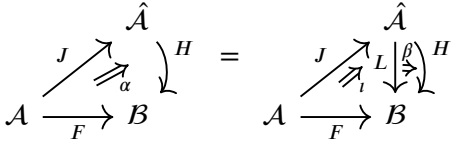
Instance: colimit (C, ι) of diagram $D : \mathbb{I} \to \mathcal{A}$

left Kan extension of D along $\underline{\star} : \mathbb{I} \to \mathbb{1}$:



extensions of D along $\underline{\star}$ amount to cocones:

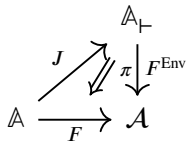
an object $A \in \mathcal{A}$ and a natural transformation $\alpha : D \to \underline{C}$.



Context category

Finite product completion $\left(\mathbb{A}_{\vdash}, \mathbb{1}, (\times), J, \pi\right)$ of a small category $\mathbb{A}$ (informally)

- (small) category $\mathbb{A}_{\vdash}$ which we call the **category of $\mathbb{A}$-contexts**;
- chosen terminal object $\mathbb{1} \in \mathbb{A}_{\vdash}$ and binary products $(\times)$;
- functor $J : \mathbb{A} \to \mathbb{A}_{\vdash}$ which we call the **embedding** of the completion;
- for every category $\mathcal{A}$ with chosen finite products and functor $F : \mathbb{A} \to \mathcal{A}$, a choice of a right Kan extension of F along J:



- moreover, this right Kan extension is a **product-preserving** functor